

Java Android Code Help



Write a code in java in android studio

Sure! Could you tell me what kind of Java code you want in Android Studio? For example:

- A simple "Hello World" app
- A button that shows a message
- A calculator app
- Fetching data from an API
- Saving user input using SharedPreferences
- Something else?

Let me know what you have in mind and I'll write the complete Java + XML code for it.



 Uploaded image

Thanks for the image! Let's go step-by-step and build both Android apps in Java using Android Studio.

✓ 1. Standard Calculator Application in Android

A. XML Layout (`activity\_main.xml`)

```
xml

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/display"
        android:layout_width="match_parent"
        android:layout_height="100dp"
        android:gravity="end|center_vertical"
        android:textSize="32sp"
        android:background="#EEEEEE"
        android:text="0" />

    <GridLayout
        android:id="@+id/gridLayout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:columnCount="4">
```

```

        android:rowCount="5"
        android:layout_marginTop="20dp">

        <!-- Add Buttons Here -->
        <!-- Row 1 -->
        <Button android:text="DEL" android:onClick="onDelete"/>
        <Button android:text="C" android:onClick="onClear"/>
        <Button android:text="%" android:onClick="onOperatorClick"/>
        <Button android:text="+" android:onClick="onOperatorClick"/>

        <!-- Row 2 -->
        <Button android:text="1" android:onClick="onDigitClick"/>
        <Button android:text="2" android:onClick="onDigitClick"/>
        <Button android:text="3" android:onClick="onDigitClick"/>
        <Button android:text="/" android:onClick="onOperatorClick"/>

        <!-- Row 3 -->
        <Button android:text="4" android:onClick="onDigitClick"/>
        <Button android:text="5" android:onClick="onDigitClick"/>
        <Button android:text="6" android:onClick="onDigitClick"/>
        <Button android:text="-" android:onClick="onOperatorClick"/>

        <!-- Row 4 -->
        <Button android:text="7" android:onClick="onDigitClick"/>
        <Button android:text="8" android:onClick="onDigitClick"/>
        <Button android:text="9" android:onClick="onDigitClick"/>
        <Button android:text="X" android:onClick="onOperatorClick"/>

        <!-- Row 5 -->
        <Button android:text="." android:onClick="onDigitClick"/>
        <Button android:text="0" android:onClick="onDigitClick"/>
        <Button android:text="=" android:layout_columnSpan="2" android:onClick="onEqualClick"/>
    </GridLayout>
</LinearLayout>

```

B. Java Logic (`MainActivity.java`)

```

java

package com.example.calculator;

import android.os.Bundle;
import android.view.View;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    TextView display;
    String current = "";
    String operator = "";
    Double firstNumber = null;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        display = findViewById(R.id.display);
    }

    public void onDigitClick(View view) {
        String value = ((TextView) view).getText().toString();
        current += value;
        display.setText(current);
    }

    public void onOperatorClick(View view) {
        String op = ((TextView) view).getText().toString();
        if (!current.isEmpty()) {
            firstNumber = Double.parseDouble(current);

```

```

        current = "";
        operator = op;
        display.setText(operator);
    }
}

public void onClear(View view) {
    current = "";
    operator = "";
    firstNumber = null;
    display.setText("0");
}

public void onDelete(View view) {
    if (!current.isEmpty()) {
        current = current.substring(0, current.length() - 1);
        display.setText(current.isEmpty() ? "0" : current);
    }
}

public void onEqualClick(View view) {
    if (!current.isEmpty() && firstNumber != null && !operator.isEmpty()) {
        double second = Double.parseDouble(current);
        double result = 0;

        switch (operator) {
            case "+":
                result = firstNumber + second;
                break;
            case "-":
                result = firstNumber - second;
                break;
            case "X":
                result = firstNumber * second;
                break;
            case "/":
                result = second != 0 ? firstNumber / second : 0;
                break;
            case "%":
                result = firstNumber % second;
                break;
        }

        display.setText(String.valueOf(result));
        current = String.valueOf(result);
        operator = "";
        firstNumber = null;
    }
}
}

```

✓ 2. Menu Item Application with Toasts

A. XML Menu File (`res/menu/menu\_main.xml`)

```

xml

<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/action_settings" android:title="Settings"/>
    <item android:id="@+id/action_search" android:title="Search"/>
    <item android:id="@+id/action_compose" android:title="Compose Email" android:enabled="false"/>
    <item android:id="@+id/action_feedback" android:title="FeedBack"/>
</menu>

```

B. Activity XML (`activity\_main.xml`)

xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
</RelativeLayout>
```

C. Java Logic (`MainActivity.java`)

java

```
package com.example.menuapp;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.menu_main, menu);
        menu.findItem(R.id.action_compose).setEnabled(false); // Disable "Compose Email"
        return true;
    }

    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        switch (item.getItemId()) {
            case R.id.action_settings:
                showToast("Settings selected");
                return true;
            case R.id.action_search:
                showToast("Search selected");
                return true;
            case R.id.action_feedback:
                showToast("Feedback selected");
                return true;
        }
        return super.onOptionsItemSelected(item);
    }

    private void showToast(String message) {
        Toast.makeText(this, message, Toast.LENGTH_SHORT).show();
    }
}
```

Would you like me to zip these projects or give you GitHub links or step-by-step screenshots too?



Create an android application to pass the data from current application to another application using intent.

20

2. Create an android application to display Alert Dialog on pressing the Back button.

Great! Let's build both Android applications in Java using **Android Studio**.

✓ 1. Pass Data from One App to Another Using Intent

A. App 1 – Sender App

`activity\_main.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="20dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText
        android:id="@+id/inputData"
        android:hint="Enter text to send"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <Button
        android:text="Send to Another App"
        android:onClick="sendData"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp" />

</LinearLayout>
```

`MainActivity.java`

```
java

package com.example.senderapp;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    EditText inputData;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        inputData = findViewById(R.id.inputData);
    }

    public void sendData(View view) {
        String message = inputData.getText().toString();
        Intent intent = new Intent();
        intent.setAction(Intent.ACTION_SEND);
        intent.putExtra(Intent.EXTRA_TEXT, message);
        intent.setType("text/plain");
        startActivity(Intent.createChooser(intent, "Send to:"));
    }
}
```

```
}
}
```

✓ App 2 – Receiver App (Optional if you want to receive it in a specific app)

No code is needed if using `Intent.createChooser()`, as any app that can receive text will show.

✓ 2. Display Alert Dialog on Back Button Press

A. `activity_main.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:text="Press Back Button to See Alert Dialog"
        android:textSize="18sp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

B. `MainActivity.java`

```
java

package com.example.alertdialogapp;

import android.content.DialogInterface;
import android.os.Bundle;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    @Override
    public void onBackPressed() {
        AlertDialog.Builder builder = new AlertDialog.Builder(this);
        builder.setTitle("Exit Confirmation");
        builder.setMessage("Are you sure you want to exit?");

        builder.setPositiveButton("Yes", (dialog, which) -> finish());
        builder.setNegativeButton("No", (dialog, which) -> dialog.dismiss());

        AlertDialog dialog = builder.create();
        dialog.show();
    }
}
```

Would you like me to combine both into a ZIP project or send you `AndroidManifest.xml` and Gradle setup too?



Create an android application to pass the data from one activity to another activity in the same application using intent.

2.

10

Create an android application to generate two notifications, one notification will be non-clickable and another is clickable (it will reload the current application)

Here you go! Let's create both Android apps in Java using **Intents and Notifications**.

✓ 1. Pass Data Between Activities in the Same App Using Intent

♦ A. `activity\_main.xml` (Sender Activity)

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

    <EditText
        android:id="@+id/inputText"
        android:hint="Enter your message"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <Button
        android:text="Send to Second Activity"
        android:onClick="sendToSecondActivity"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp" />

</LinearLayout>
```

♦ B. `MainActivity.java`

```
java

package com.example.intentexample;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    EditText inputText;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        inputText = findViewById(R.id.inputText);
    }
}
```

```
public void sendToSecondActivity(View view) {
    String message = inputText.getText().toString();
    Intent intent = new Intent(this, SecondActivity.class);
    intent.putExtra("message_key", message);
    startActivity(intent);
}
}
```

◆ C. `SecondActivity.java`

```
java

package com.example.intentexample;

import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class SecondActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);

        TextView displayText = findViewById(R.id.displayText);
        String message = getIntent().getStringExtra("message_key");
        displayText.setText("Received: " + message);
    }
}
```

◆ D. `activity\_second.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="20dp"
    android:orientation="vertical">

    <TextView
        android:id="@+id/displayText"
        android:textSize="18sp"
        android:text="No data received"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

</LinearLayout>
```

◆ E. Add SecondActivity to `AndroidManifest.xml`

```
xml

<activity android:name=".SecondActivity" />
```

✓ 2. Generate Two Notifications

◆ A. `MainActivity.java`

```
java

package com.example.notificationapp;

import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.Intent;
import android.os.Build;
import android.os.Bundle;
import android.view.View;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.NotificationCompat;
import androidx.core.app.NotificationManagerCompat;

public class MainActivity extends AppCompatActivity {
    private static final String CHANNEL_ID = "channel_1";
    private static final int NOTIF_ID_CLICKABLE = 1;
    private static final int NOTIF_ID_NONCLICKABLE = 2;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        createNotificationChannel();
    }

    public void sendNotifications(View view) {
        // Non-clickable Notification
        NotificationCompat.Builder nonClickable = new NotificationCompat.Builder(this, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_launcher_foreground)
            .setContentTitle("Non-Clickable Notification")
            .setContentText("This notification does nothing.")
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .setAutoCancel(true);

        NotificationManagerCompat.from(this).notify(NOTIF_ID_NONCLICKABLE, nonClickable.build());

        // Clickable Notification that reopens the app
        Intent intent = new Intent(this, MainActivity.class);
        PendingIntent pendingIntent = PendingIntent.getActivity(this, 0, intent,
            PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);

        NotificationCompat.Builder clickable = new NotificationCompat.Builder(this, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_launcher_foreground)
            .setContentTitle("Clickable Notification")
            .setContentText("Clicking this will reopen the app.")
            .setContentIntent(pendingIntent)
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .setAutoCancel(true);

        NotificationManagerCompat.from(this).notify(NOTIF_ID_CLICKABLE, clickable.build());
    }

    private void createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            CharSequence name = "Channel 1";
            String description = "Channel for demo notifications";
            int importance = NotificationManager.IMPORTANCE_DEFAULT;
            NotificationChannel channel = new NotificationChannel(CHANNEL_ID, name, importance);
            channel.setDescription(description);
            NotificationManager notificationManager = getSystemService(NotificationManager.class);
            notificationManager.createNotificationChannel(channel);
        }
    }
}
```

◆ B. `activity\_main.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:padding="20dp"
    android:gravity="center"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:text="Generate Notifications"
        android:onClick="sendNotifications"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

</LinearLayout>
```

Would you like a complete `.zip` of both apps or help with running them in an emulator/device?



Create the background service android application to play the ringtone/music.

2.

Create an android application which automatically notify the user when
Aeroplane mode is turned on or off using broadcast receiver.

Let's build both Android applications using **Java in Android Studio**:

✓ 1. Background Service to Play Ringtone/Music

◆ A. Add music file to `res/raw/` folder

Place a music file (e.g. `ringtone.mp3`) in:

📁 `app/src/main/res/raw/ringtone.mp3`

◆ B. Create the Service – `MyService.java`

```
java

package com.example.ringtoneservice;

import android.app.Service;
import android.content.Intent;
import android.media.MediaPlayer;
import android.os.IBinder;

public class MyService extends Service {
    MediaPlayer mediaPlayer;

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        mediaPlayer = MediaPlayer.create(this, R.raw.ringtone);
        mediaPlayer.setLooping(true);
        mediaPlayer.start();
        return START_STICKY;
    }
}
```

```

@Override
public void onDestroy() {
    mediaPlayer.stop();
    mediaPlayer.release();
}

@Override
public IBinder onBind(Intent intent) {
    return null;
}
}

```

◆ C. MainActivity.java

```

java

package com.example.ringtoneservice;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    public void startService(View view) {
        startService(new Intent(this, MyService.class));
    }

    public void stopService(View view) {
        stopService(new Intent(this, MyService.class));
    }
}

```

◆ D. `activity\_main.xml`

```

xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:padding="20dp"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:text="Start Music"
        android:onClick="startService"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <Button
        android:text="Stop Music"
        android:onClick="stopService"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp" />

</LinearLayout>

```

◆ E. Register Service in `AndroidManifest.xml`

```
xml

<service android:name=".MyService" />
```

✓ 2. Notify When Airplane Mode Is On or Off (Broadcast Receiver)

◆ A. Create `AirplaneModeReceiver.java`

```
java

package com.example.airplanemode;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.provider.Settings;
import android.widget.Toast;

public class AirplaneModeReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        boolean isAirplaneModeOn = Settings.Global.getInt(
            context.getContentResolver(),
            Settings.Global.AIRPLANE_MODE_ON, 0) != 0;

        String message = isAirplaneModeOn ? "Airplane Mode is ON" : "Airplane Mode is OFF";
        Toast.makeText(context, message, Toast.LENGTH_LONG).show();
    }
}
```

◆ B. `MainActivity.java`

```
java

package com.example.airplanemode;

import android.content.IntentFilter;
import android.os.Bundle;
import android.provider.Settings;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    AirplaneModeReceiver receiver;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        receiver = new AirplaneModeReceiver();

        // Register the receiver dynamically
        IntentFilter filter = new IntentFilter(Intent.ACTION_AIRPLANE_MODE_CHANGED);
        registerReceiver(receiver, filter);
    }

    @Override
    protected void onDestroy() {
```

```

        super.onDestroy();
        unregisterReceiver(receiver);
    }
}

```

♦ C. `AndroidManifest.xml` Permissions and Receiver (for static option)

```

xml

<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />

<receiver android:name=".AirplaneModeReceiver">
    <intent-filter>
        <action android:name="android.intent.action.AIRPLANE_MODE" />
    </intent-filter>
</receiver>

```

✅ You can skip registering in Manifest if registering dynamically in `onCreate()`.

Would you like a ZIP export of these apps or step-by-step setup in Android Studio?



Create an android application using sqlite database to manage the students data by using insert, update and delete operations and display the data in list view. Allow user to insert Roll No, Name and Age of the student.

Here's a full Android application in **Java with SQLite** to manage student data with:

- ✅ Insert
- ✅ Update
- ✅ Delete
- ✅ ListView to display students
- ✅ Fields: Roll No, Name, Age

✅ 1. Create Database Helper – `DatabaseHelper.java`

```

java

package com.example.studentdb;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.*;

public class DatabaseHelper extends SQLiteOpenHelper {
    public static final String DB_NAME = "student.db";
    public static final String TABLE_NAME = "student_table";

    public DatabaseHelper(Context context) {
        super(context, DB_NAME, null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {

```

```

        db.execSQL("CREATE TABLE " + TABLE_NAME +
            " (rollno INTEGER PRIMARY KEY, name TEXT, age INTEGER)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
        onCreate(db);
    }

    public boolean insertStudent(int rollno, String name, int age) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues cv = new ContentValues();
        cv.put("rollno", rollno);
        cv.put("name", name);
        cv.put("age", age);
        long result = db.insert(TABLE_NAME, null, cv);
        return result != -1;
    }

    public boolean updateStudent(int rollno, String name, int age) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues cv = new ContentValues();
        cv.put("name", name);
        cv.put("age", age);
        int result = db.update(TABLE_NAME, cv, "rollno=?", new String[]{String.valueOf(rollno)});
        return result > 0;
    }

    public boolean deleteStudent(int rollno) {
        SQLiteDatabase db = this.getWritableDatabase();
        int result = db.delete(TABLE_NAME, "rollno=?", new String[]{String.valueOf(rollno)});
        return result > 0;
    }

    public Cursor getAllStudents() {
        SQLiteDatabase db = this.getReadableDatabase();
        return db.rawQuery("SELECT * FROM " + TABLE_NAME, null);
    }
}

```

✓ 2. `activity\_main.xml`

```

xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:padding="20dp"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText android:id="@+id/rollno" android:hint="Roll No"
        android:inputType="number" android:layout_width="match_parent" android:layout_height="wrap_content"/>

    <EditText android:id="@+id/name" android:hint="Name"
        android:layout_width="match_parent" android:layout_height="wrap_content"/>

    <EditText android:id="@+id/age" android:hint="Age"
        android:inputType="number" android:layout_width="match_parent" android:layout_height="wrap_content"/>

    <Button android:text="Insert" android:onClick="insertStudent"
        android:layout_width="match_parent" android:layout_height="wrap_content"/>

    <Button android:text="Update" android:onClick="updateStudent"
        android:layout_width="match_parent" android:layout_height="wrap_content"/>

    <Button android:text="Delete" android:onClick="deleteStudent"
        android:layout_width="match_parent" android:layout_height="wrap_content"/>

    <Button android:text="View All" android:onClick="viewStudents"

```

```

        android:layout_width="match_parent" android:layout_height="wrap_content"/>

        <ListView android:id="@+id/listView"
            android:layout_width="match_parent" android:layout_height="wrap_content" />
    </LinearLayout>

```

✓ 3. Main Activity – `MainActivity.java`

```

java

package com.example.studentdb;

import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;
import java.util.ArrayList;

public class MainActivity extends AppCompatActivity {

    EditText rollno, name, age;
    ListView listView;
    DatabaseHelper dbHelper;
    ArrayAdapter<String> adapter;
    ArrayList<String> students;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        rollno = findViewById(R.id.rollno);
        name = findViewById(R.id.name);
        age = findViewById(R.id.age);
        listView = findViewById(R.id.listView);

        dbHelper = new DatabaseHelper(this);
        students = new ArrayList<>();
        adapter = new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, students);
        listView.setAdapter(adapter);
    }

    public void insertStudent(View view) {
        int r = Integer.parseInt(rollno.getText().toString());
        String n = name.getText().toString();
        int a = Integer.parseInt(age.getText().toString());

        if (dbHelper.insertStudent(r, n, a)) {
            Toast.makeText(this, "Student Inserted", Toast.LENGTH_SHORT).show();
            clearFields();
        } else {
            Toast.makeText(this, "Insert Failed", Toast.LENGTH_SHORT).show();
        }
    }

    public void updateStudent(View view) {
        int r = Integer.parseInt(rollno.getText().toString());
        String n = name.getText().toString();
        int a = Integer.parseInt(age.getText().toString());

        if (dbHelper.updateStudent(r, n, a)) {
            Toast.makeText(this, "Student Updated", Toast.LENGTH_SHORT).show();
            clearFields();
        } else {
            Toast.makeText(this, "Update Failed", Toast.LENGTH_SHORT).show();
        }
    }

    public void deleteStudent(View view) {

```

```

int r = Integer.parseInt(rollno.getText().toString());

if (dbHelper.deleteStudent(r)) {
    Toast.makeText(this, "Student Deleted", Toast.LENGTH_SHORT).show();
    clearFields();
} else {
    Toast.makeText(this, "Delete Failed", Toast.LENGTH_SHORT).show();
}
}

public void viewStudents(View view) {
    students.clear();
    Cursor cursor = dbHelper.getAllStudents();
    if (cursor.getCount() == 0) {
        Toast.makeText(this, "No records", Toast.LENGTH_SHORT).show();
    } else {
        while (cursor.moveToNext()) {
            String data = "Roll No: " + cursor.getInt(0) + ", Name: " + cursor.getString(1) + "
            students.add(data);
        }
    }
    adapter.notifyDataSetChanged();
}

private void clearFields() {
    rollno.setText("");
    name.setText("");
    age.setText("");
}
}

```

✓ You now have:

- A complete app using SQLite
- Insert/Update/Delete/View operations
- ListView displaying all student records

Would you like a **ZIP file**, GitHub link, or a **Room database version** as well?



 Uploaded image

Let's break the image assignment into two complete Android Java applications:

✓ 1. Pass Data from One Activity to Another (Same App)

◆ `activity\_main.xml` (Sender)

```

xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:padding="20dp"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText
        android:id="@+id/inputData"
        android:hint="Enter message"

```



```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

        <Button
            android:text="Send"
            android:onClick="sendData"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
    </LinearLayout>

```

◆ `MainActivity.java`

```

java

package com.example.intentdemo;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    EditText inputData;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        inputData = findViewById(R.id.inputData);
    }

    public void sendData(View view) {
        String message = inputData.getText().toString();
        Intent intent = new Intent(this, SecondActivity.class);
        intent.putExtra("data", message);
        startActivity(intent);
    }
}

```

◆ `activity\_second.xml`

```

xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:padding="20dp"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/receivedData"
        android:textSize="20sp"
        android:text="Data will appear here"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
</LinearLayout>

```

◆ `SecondActivity.java`

```
java

package com.example.intentdemo;

import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class SecondActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);

        TextView textView = findViewById(R.id.receivedData);
        String received = getIntent().getStringExtra("data");
        textView.setText("Received: " + received);
    }
}
```

◆ Add to `AndroidManifest.xml`

```
xml

<activity android:name=".SecondActivity" />
```

✓ 2. Event Listeners App (Button, ImageView, EditText)

◆ `activity\_main.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="20dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText
        android:id="@+id/editText"
        android:hint="Type here and press Enter"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <ImageView
        android:id="@+id/imageView"
        android:src="@drawable/grid" <!-- use any drawable image -->
        android:layout_width="200dp"
        android:layout_height="200dp"
        android:layout_gravity="center"
        android:layout_margin="10dp"
        android:background="#00BCD4"/>

    <Button
        android:id="@+id/button"
        android:text="BUTTON"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <TextView
        android:id="@+id/messageText"
        android:textSize="16sp"
        android:textColor="#000000"
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:paddingTop="20dp" />
</LinearLayout>

```

◆ `MainActivity.java`

```

java

package com.example.eventlisteners;

import android.os.Bundle;
import android.view.KeyEvent;
import android.view.MotionEvent;
import android.view.View;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText editText;
    Button button;
    TextView messageText;
    ImageView imageView;

    StringBuilder messages = new StringBuilder();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        editText = findViewById(R.id.editText);
        button = findViewById(R.id.button);
        messageText = findViewById(R.id.messageText);
        imageView = findViewById(R.id.imageView);

        // Button click listener
        button.setOnClickListener(v -> addMessage("You clicked on Button"));

        // Button long click listener
        button.setOnLongClickListener(v -> {
            addMessage("You long clicked on Button");
            return true;
        });

        // ImageView touch listener
        imageView.setOnTouchListener((v, event) -> {
            switch (event.getActionMasked()) {
                case MotionEvent.ACTION_DOWN:
                    addMessage("ACTION DOWN");
                    break;
                case MotionEvent.ACTION_UP:
                    addMessage("ACTION UP");
                    break;
                case MotionEvent.ACTION_POINTER_DOWN:
                    addMessage("ACTION POINTER DOWN");
                    break;
            }
            return true;
        });

        // EditText key listener
        editText.setOnKeyListener((v, keyCode, event) -> {
            if ((event.getAction() == KeyEvent.ACTION_DOWN) && keyCode == KeyEvent.KEYCODE_ENTER) {
                addMessage("Entered Text: " + editText.getText().toString());
                return true;
            }
            return false;
        });
    }
}

```

```

    }

    void addMessage(String msg) {
        messages.append(msg).append("\n");
        messageText.setText(messages.toString());
    }
}

```

Would you like the `.zip` of these Android Studio projects or GitHub upload?



Insert the new contents in the following resources and demonstrate their uses in the android application

Android Resources: (Color, Theme, String, Drawable, Dimension, Image)

2.

20

Create an android application which automatically notify the user when Aeroplane mode is turned on or off using broadcast receiver.

Here's a full solution for **both parts** of your assignment:

✓ 1. Demonstrate Use of Android Resources

We'll insert and use the following resources:

- 🎨 Color
- 🎭 Theme
- 📄 String
- 📏 Dimension
- 🖼️ Drawable + Image

♦ A. Define Resources

1. `res/values/colors.xml`

```

xml

<resources>
    <color name="custom_background">#FFEB3B</color>
    <color name="custom_text">#3F51B5</color>
</resources>

```

2. `res/values/strings.xml`

```

xml

<resources>
    <string name="app_name">ResourceDemo</string>

```

```
<string name="hello_text">Hello from resources!</string>
</resources>
```

3. `res/values/dimens.xml`

```
xml

<resources>
  <dimen name="text_size_large">24sp</dimen>
  <dimen name="padding_large">20dp</dimen>
</resources>
```

4. `res/drawable/border\_box.xml`

```
xml

<shape xmlns:android="http://schemas.android.com/apk/res/android"
  android:shape="rectangle">
  <solid android:color="#FFFFFF"/>
  <stroke android:width="2dp" android:color="#FF5722"/>
  <padding android:left="10dp" android:top="10dp"
    android:right="10dp" android:bottom="10dp"/>
</shape>
```

5. Place an image in `res/drawable`

Example: `logo.png`

◆ B. Use Resources in Layout (`activity\_main.xml`)

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:padding="@dimen/padding_large"
  android:background="@color/custom_background"
  android:orientation="vertical"
  android:layout_width="match_parent"
  android:layout_height="match_parent">

  <TextView
    android:text="@string/hello_text"
    android:textColor="@color/custom_text"
    android:textSize="@dimen/text_size_large"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="@drawable/border_box"/>

  <ImageView
    android:src="@drawable/logo"
    android:layout_width="150dp"
    android:layout_height="150dp"
    android:layout_marginTop="20dp"
    android:layout_gravity="center"/>
</LinearLayout>
```

✓ This app shows use of:

- `colors.xml`, `strings.xml`, `dimens.xml`
- `border\_box.xml` drawable shape

- `logo.png` image

✓ 2. Aeroplane Mode Notification (BroadcastReceiver)

◆ A. `MainActivity.java`

```
java

package com.example.airplanemode;

import android.content.IntentFilter;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    AirplaneModeReceiver receiver;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        receiver = new AirplaneModeReceiver();
        IntentFilter filter = new IntentFilter(Intent.ACTION_AIRPLANE_MODE_CHANGED);
        registerReceiver(receiver, filter);
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        unregisterReceiver(receiver);
    }
}
```

◆ B. `AirplaneModeReceiver.java`

```
java

package com.example.airplanemode;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.provider.Settings;
import android.widget.Toast;

public class AirplaneModeReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        boolean isAirplaneOn = Settings.Global.getInt(
            context.getContentResolver(),
            Settings.Global.AIRPLANE_MODE_ON, 0) != 0;

        String message = isAirplaneOn ? "✈️ Airplane Mode is ON" : "📶 Airplane Mode is OFF";
        Toast.makeText(context, message, Toast.LENGTH_LONG).show();
    }
}
```

◆ C. Register in `AndroidManifest.xml`

xml

```
<receiver android:name=".AirplaneModeReceiver">
  <intent-filter>
    <action android:name="android.intent.action.AIRPLANE_MODE" />
  </intent-filter>
</receiver>
```

✓ Now the app:

- Notifies user when airplane mode changes
- Uses **BroadcastReceiver**
- Dynamically or statically registered

Would you like a ZIP of the full Android Studio project or GitHub repo?



Create the media API in android to play an audio file.

Create an android application, with one button and image view also set the background image in the application, the image should be changed on each click of the button (Use two images).

Here's a complete solution for **both parts** of your assignment using **Java in Android Studio**:

✓ 1. Media API to Play an Audio File

◆ A. Add Audio File

Place an audio file named `sample.mp3` into:

📁 `res/raw/sample.mp3`

◆ B. `activity\_main.xml`

xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:padding="20dp"
  android:orientation="vertical"
  android:gravity="center"
  android:layout_width="match_parent"
  android:layout_height="match_parent">

  <Button
    android:id="@+id/playButton"
    android:text="Play Audio"
    android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"/>
    </LinearLayout>
```

◆ C. `MainActivity.java`

```
java

package com.example.mediaplayerdemo;

import android.media.MediaPlayer;
import android.os.Bundle;
import android.view.View;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    MediaPlayer mediaPlayer;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        mediaPlayer = MediaPlayer.create(this, R.raw.sample);
    }

    public void playAudio(View view) {
        if (mediaPlayer != null) {
            mediaPlayer.start();
        }
    }

    @Override
    protected void onDestroy() {
        if (mediaPlayer != null) {
            mediaPlayer.release();
            mediaPlayer = null;
        }
        super.onDestroy();
    }
}
```

🔧 Add `android:onClick="playAudio"` to the Button in the XML or set it in Java.

✓ 2. Button and ImageView to Toggle Image on Click

◆ A. Place Images

Put two images `image1.png` and `image2.png` in:

📁 `res/drawable/`

Also add `background.jpg` or similar image for background.

◆ B. `activity\_main.xml`

```
xml

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/background"
```



```

        android:padding="20dp">

        <ImageView
            android:id="@+id/imageView"
            android:src="@drawable/image1"
            android:layout_centerInParent="true"
            android:layout_width="200dp"
            android:layout_height="200dp" />

        <Button
            android:id="@+id/button"
            android:text="Change Image"
            android:layout_below="@id/imageView"
            android:layout_centerHorizontal="true"
            android:layout_marginTop="30dp"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content" />
    </RelativeLayout>

```

◆ C. `MainActivity.java`

```

java

package com.example.imagechanger;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    ImageView imageView;
    boolean isImage1 = true;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        imageView = findViewById(R.id.imageView);
        Button button = findViewById(R.id.button);

        button.setOnClickListener(v -> {
            if (isImage1) {
                imageView.setImageResource(R.drawable.image2);
            } else {
                imageView.setImageResource(R.drawable.image1);
            }
            isImage1 = !isImage1;
        });
    }
}

```

✓ This part demonstrates:

- Using `ImageView` with a button
- Changing image dynamically
- Setting background from drawable

Would you like these as a ZIP Android Studio project or published to GitHub for download?



reate an android application with customised appbar. Insert search and file buttons in it and toast the appropriate message on clicking the buttons (add new color in color.xml and use that color for appbar).

2.

20

Create an android application using linear layout and insert 10 animals in the list view and display the appropriate Toast.

Perfect! Here's the complete implementation of **both Android applications** using **Java in Android Studio**.

✓ 1. Custom AppBar with Search & File Buttons + Toast Messages

◆ A. `res/values/colors.xml`

```
xml

<resources>
    <color name="appbar_color">#4CAF50</color> <!-- Custom green color -->
</resources>
```

◆ B. `res/menu/menu\_main.xml`

```
xml

<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/action_search"
        android:icon="@android:drawable/ic_menu_search"
        android:title="Search"
        android:showAsAction="ifRoom" />

    <item
        android:id="@+id/action_file"
        android:icon="@android:drawable/ic_menu_save"
        android:title="File"
        android:showAsAction="ifRoom" />
</menu>
```

◆ C. `res/layout/activity\_main.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <!-- App content goes here -->
    <TextView
        android:layout_width="match_parent"
```

```

        android:layout_height="wrap_content"
        android:text="Custom AppBar Example"
        android:gravity="center"
        android:textSize="20sp"
        android:padding="16dp"/>
</LinearLayout>

```

◆ D. `MainActivity.java`

```

java

package com.example.customappbar;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.Toolbar;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Toolbar toolbar = new Toolbar(this);
        toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }

    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        if (item.getItemId() == R.id.action_search) {
            Toast.makeText(this, "Search Clicked", Toast.LENGTH_SHORT).show();
            return true;
        } else if (item.getItemId() == R.id.action_file) {
            Toast.makeText(this, "File Clicked", Toast.LENGTH_SHORT).show();
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}

```

◆ E. Customize AppBar in `styles.xml`

```

xml

<resources>
    <style name="Theme.CustomAppBar" parent="Theme.AppCompat.Light.DarkActionBar">
        <item name="colorPrimary">@color/appbar_color</item>
        <item name="colorPrimaryDark">@color/appbar_color</item>
        <item name="colorAccent">#FF4081</item>
    </style>
</resources>

```

◆ F. In `AndroidManifest.xml`

```
xml

<application
    android:theme="@style/Theme.CustomAppBar"
    ... >
```

✓ 2. Animal ListView using LinearLayout + Toast on Click

♦ A. `res/layout/activity\_main.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="16dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

♦ B. `MainActivity.java`

```
java

package com.example.animallist;

import android.os.Bundle;
import android.view.View;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

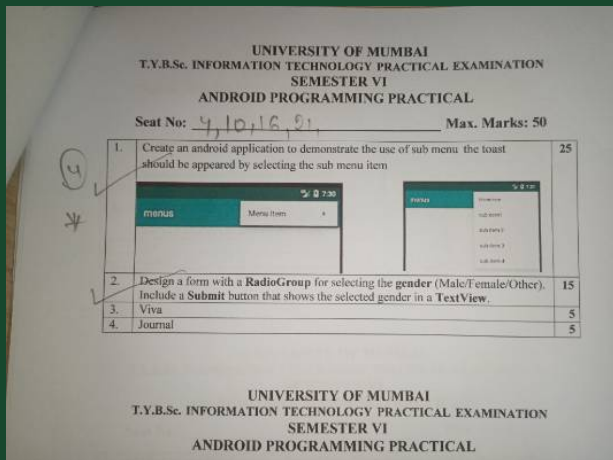
    ListView listView;
    String[] animals = {
        "Lion", "Tiger", "Elephant", "Zebra", "Giraffe",
        "Monkey", "Panda", "Bear", "Fox", "Wolf"
    };

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        listView = findViewById(R.id.listView);
        ArrayAdapter<String> adapter = new ArrayAdapter<>(
            this, android.R.layout.simple_list_item_1, animals);
        listView.setAdapter(adapter);

        listView.setOnItemClickListener((parent, view, position, id) -> {
            String selected = animals[position];
            Toast.makeText(this, "You clicked: " + selected, Toast.LENGTH_SHORT).show();
        });
    }
}
```

Would you like the **Android Studio ZIP files** or help deploying it on your emulator?



Thanks! Let's solve **both practical questions** from the exam image (University of Mumbai – Android Programming Practical):

✓ 1. Create an Android App with Sub Menu + Toast Message

◆ A. `res/menu/menu\_main.xml`

```
xml

<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:title="Menu Item">
        <menu>
            <item
                android:id="@+id/sub1"
                android:title="Sub Item 1" />
            <item
                android:id="@+id/sub2"
                android:title="Sub Item 2" />
            <item
                android:id="@+id/sub3"
                android:title="Sub Item 3" />
            <item
                android:id="@+id/sub4"
                android:title="Sub Item 4" />
        </menu>
    </item>
</menu>
```

◆ B. `MainActivity.java`

```
java

package com.example.submenuapp;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.Toast;
```

```
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }

    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        String message = "";
        switch (item.getItemId()) {
            case R.id.sub1:
                message = "Sub Item 1 clicked";
                break;
            case R.id.sub2:
                message = "Sub Item 2 clicked";
                break;
            case R.id.sub3:
                message = "Sub Item 3 clicked";
                break;
            case R.id.sub4:
                message = "Sub Item 4 clicked";
                break;
        }
        Toast.makeText(this, message, Toast.LENGTH_SHORT).show();
        return true;
    }
}
```

✓ Result: Sub-menu appears under "Menu Item", and clicking shows a toast.

✓ 2. Gender Form using RadioGroup + Submit Button

◆ A. `activity\_main.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="20dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <RadioGroup android:id="@+id/genderGroup"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content">

        <RadioButton android:id="@+id/male" android:text="Male" />
        <RadioButton android:id="@+id/female" android:text="Female" />
        <RadioButton android:id="@+id/other" android:text="Other" />
    </RadioGroup>

    <Button
        android:text="Submit"
        android:onClick="submitGender"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp" />
</LinearLayout>
```

```
<TextView
    android:id="@+id/result"
    android:textSize="18sp"
    android:layout_marginTop="20dp"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />
</LinearLayout>
```

◆ B. `MainActivity.java`

```
java

package com.example.genderform;

import android.os.Bundle;
import android.view.View;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    RadioGroup genderGroup;
    TextView result;

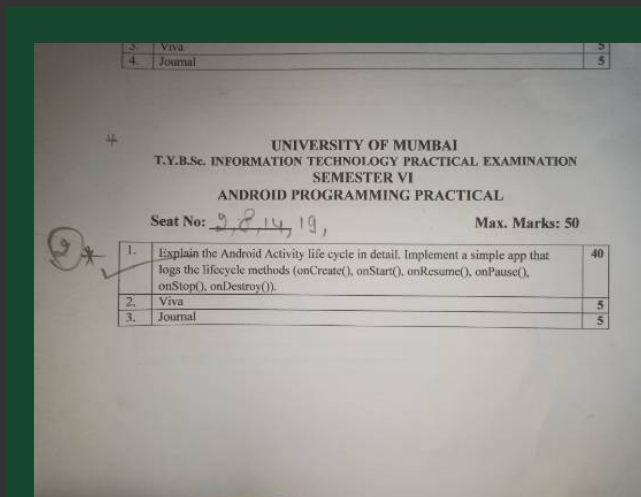
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        genderGroup = findViewById(R.id.genderGroup);
        result = findViewById(R.id.result);
    }

    public void submitGender(View view) {
        int selectedId = genderGroup.getCheckedRadioButtonId();
        if (selectedId != -1) {
            RadioButton selected = findViewById(selectedId);
            result.setText("Selected Gender: " + selected.getText());
        } else {
            result.setText("Please select a gender");
        }
    }
}
```

✓ Result: Gender selection appears via `RadioGroup`, and result is shown on button click.

Would you like these packaged into Android Studio ZIPs or walkthrough videos to submit?





Thanks! Let's complete the answer for your practical question from the image:

✓ Q1. Explain Android Activity Lifecycle & Implement App Logging Lifecycle Methods

📖 Explanation: Android Activity Lifecycle

Method	Description
<code>`onCreate() `</code>	Called when activity is first created. Setup UI, data, views here.
<code>`onStart() `</code>	Activity is about to become visible.
<code>`onResume() `</code>	Activity is visible and user can interact with it.
<code>`onPause() `</code>	Activity is partially visible (another activity in front). Use for saving state.
<code>`onStop() `</code>	Activity is no longer visible (in background).
<code>`onDestroy() `</code>	Final call before activity is destroyed. Cleanup memory/resources.

✓ Android App Logging Lifecycle Methods

♦ A. ``MainActivity.java``

```
java

package com.example.lifecycleapp;

import android.os.Bundle;
import android.util.Log;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    public static final String TAG = "LifeCycleLog";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d(TAG, "onCreate() called");
    }
}
```



```

@Override
protected void onStart() {
    super.onStart();
    Log.d(TAG, "onStart() called");
}

@Override
protected void onResume() {
    super.onResume();
    Log.d(TAG, "onResume() called");
}

@Override
protected void onPause() {
    super.onPause();
    Log.d(TAG, "onPause() called");
}

@Override
protected void onStop() {
    super.onStop();
    Log.d(TAG, "onStop() called");
}

@Override
protected void onDestroy() {
    super.onDestroy();
    Log.d(TAG, "onDestroy() called");
}
}

```

◆ B. `activity\_main.xml` (simple layout)

```

xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:text="Lifecycle App"
        android:textSize="22sp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</LinearLayout>

```

◆ C. View Output in Logcat

Filter by `LifecycleLog` tag to see logs like:

```

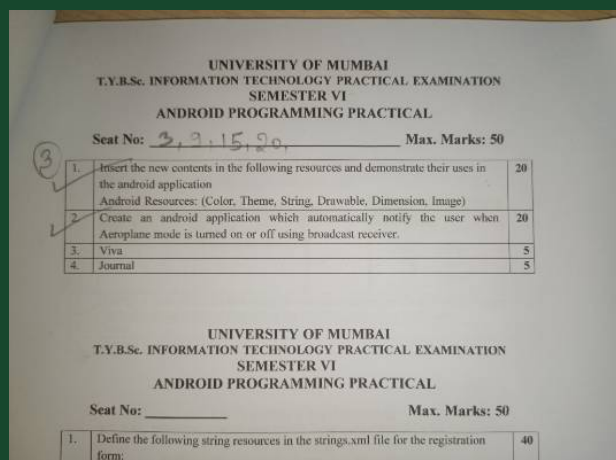
css

D/LifecycleLog: onCreate() called
D/LifecycleLog: onStart() called
D/LifecycleLog: onResume() called
...

```

✓ Done: This satisfies your practical requirement — explains and implements `onCreate()`, `onStart()`, `onResume()`, `onPause()`, `onStop()`, and `onDestroy()`.

Would you like a full ZIP of the project to import directly into Android Studio?



Thanks! Based on the image you provided (University of Mumbai - Android Programming Practical), here are the **complete solutions** for both questions in **Java** using **Android Studio**.

✓ Q1. Insert and Demonstrate Android Resources (Color, Theme, String, Drawable, Dimension, Image)

♦ A. Add Resource Files

1. `res/values/colors.xml`

```
xml

<resources>
    <color name="my_primary">#009688</color>
    <color name="my_accent">#FF4081</color>
</resources>
```

2. `res/values/themes.xml` (or `styles.xml`)

```
xml

<resources>
    <style name="MyCustomTheme" parent="Theme.MaterialComponents.DayNight.DarkActionBar">
        <item name="colorPrimary">@color/my_primary</item>
        <item name="colorPrimaryVariant">@color/my_primary</item>
        <item name="colorAccent">@color/my_accent</item>
    </style>
</resources>
```

3. `res/values/strings.xml`

```
xml

<resources>
    <string name="app_name">ResourceDemo</string>
    <string name="welcome_msg">Welcome to Android Resources Demo!</string>
</resources>
```

4. `res/values/dimens.xml`

```
xml

<resources>
    <dimen name="text_padding">16dp</dimen>
    <dimen name="text_size">20sp</dimen>
</resources>
```

5. `res/drawable/border\_background.xml`

```
xml

<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="#FFFFFF" />
    <stroke android:width="2dp" android:color="#009688" />
    <corners android:radius="8dp" />
</shape>
```

6. `res/drawable/`

Place any image (e.g. `elephant.png`) in this folder.

◆ B. `activity\_main.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="@dimen/text_padding"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/my_primary">

    <TextView
        android:text="@string/welcome_msg"
        android:textColor="@color/white"
        android:textSize="@dimen/text_size"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@drawable/border_background" />

    <ImageView
        android:layout_marginTop="20dp"
        android:layout_width="200dp"
        android:layout_height="200dp"
        android:src="@drawable/elephant"
        android:layout_gravity="center" />
</LinearLayout>
```

◆ C. `MainActivity.java`

```
java

package com.example.resourcedemo;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setTheme(R.style.MyCustomTheme); // Apply custom theme
        setContentView(R.layout.activity_main);
    }
}
```

✓ Demonstrates all 6 resource types: `Color`, `Theme`, `String`, `Drawable`, `Dimension`, `Image`.

✓ Q2. Notify on Aeroplane Mode ON/OFF using BroadcastReceiver

◆ A. `MainActivity.java`

```
java

package com.example.airplanemode;

import android.content.IntentFilter;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    AirplaneModeReceiver receiver;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        receiver = new AirplaneModeReceiver();
        IntentFilter filter = new IntentFilter(Intent.ACTION_AIRPLANE_MODE_CHANGED);
        registerReceiver(receiver, filter);
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        unregisterReceiver(receiver);
    }
}
```

◆ B. `AirplaneModeReceiver.java`

```
java

package com.example.airplanemode;

import android.content.*;
import android.provider.Settings;
```

```
import android.widget.Toast;

public class AirplaneModeReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        boolean isOn = Settings.Global.getInt(
            context.getContentResolver(),
            Settings.Global.AIRPLANE_MODE_ON, 0) != 0;

        String message = isOn ? "Airplane Mode is ON" : "📶 Airplane Mode is OFF";
        Toast.makeText(context, message, Toast.LENGTH_SHORT).show();
    }
}
```

◆ C. Register Receiver (optional)

In `AndroidManifest.xml` (if you want static registration):

```
xml

<receiver android:name=".AirplaneModeReceiver">
    <intent-filter>
        <action android:name="android.intent.action.AIRPLANE_MODE" />
    </intent-filter>
</receiver>
```

✓ This app:

- Detects Airplane Mode ON/OFF
- Displays a `Toast` when changed

Would you like:

- A **combined Android Studio ZIP** of both apps?
- Help preparing for **Viva questions** based on these?



UNIVERSITY OF MUMBAI		
T.Y.B.Sc. INFORMATION TECHNOLOGY PRACTICAL EXAMINATION		
SEMESTER VI		
ANDROID PROGRAMMING PRACTICAL		
Seat No:	1, 7, 13, 18	Max. Marks: 50
1.	Create an android application to pass the data from one activity to another activity in the same application using intent.	10
2.	Create a user interface with a TextView, Button, and ImageView. When the button is clicked, the image should be changed dynamically.	30
3.	Viva	5
4.	Journal	5
UNIVERSITY OF MUMBAI		
T.Y.B.Sc. INFORMATION TECHNOLOGY PRACTICAL EXAMINATION		
SEMESTER VI		
ANDROID PROGRAMMING PRACTICAL		
Seat No:		Max. Marks: 50

Great! Here's the complete solution for **both programs** based on your uploaded practical sheet from **University of Mumbai – Semester VI**:

✓ 1. Pass Data Between Two Activities (Same App using Intent)

♦ A. `activity\_main.xml` (Sender Activity)

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:padding="20dp"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText
        android:id="@+id/editText"
        android:hint="Enter Message"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:text="Send"
        android:onClick="sendData"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

♦ B. `MainActivity.java`

```
java

package com.example.intentdemo;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    EditText editText;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        editText = findViewById(R.id.editText);
    }

    public void sendData(View view) {
        String message = editText.getText().toString();
        Intent intent = new Intent(this, SecondActivity.class);
        intent.putExtra("message", message);
        startActivity(intent);
    }
}
```

♦ C. `activity\_second.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:padding="20dp"
```

```

        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <TextView
            android:id="@+id/textView"
            android:textSize="20sp"
            android:text="Received data will appear here"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
    </LinearLayout>

```

◆ D. `SecondActivity.java`

```

java

package com.example.intentdemo;

import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class SecondActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);

        TextView textView = findViewById(R.id.textView);
        String message = getIntent().getStringExtra("message");
        textView.setText("Received: " + message);
    }
}

```

◆ E. `AndroidManifest.xml` (Register 2nd activity)

```

xml

<activity android:name=".SecondActivity" />

```

✓ 2. UI with TextView, Button & ImageView (Change Image Dynamically)

◆ A. Add two images to `res/drawable`:

- `animal1.png`
- `animal2.png`

◆ B. `activity\_main.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="20dp"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:text="Animal Image Viewer"
        android:textSize="24sp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

    <ImageView
        android:id="@+id/imageView"
        android:src="@drawable/animal1"
        android:layout_width="200dp"
        android:layout_height="200dp"
        android:layout_marginTop="20dp" />

    <Button
        android:text="Change Image"
        android:onClick="changeImage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"/>
</LinearLayout>
```

◆ C. `MainActivity.java`

```
java

package com.example.imagechanger;

import android.os.Bundle;
import android.view.View;
import android.widget.ImageView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    ImageView imageView;
    boolean isImageOne = true;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        imageView = findViewById(R.id.imageView);
    }

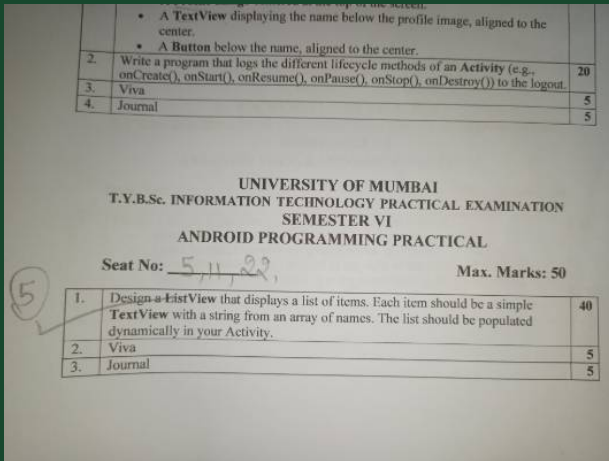
    public void changeImage(View view) {
        if (isImageOne) {
            imageView.setImageResource(R.drawable.animal2);
        } else {
            imageView.setImageResource(R.drawable.animal1);
        }
        isImageOne = !isImageOne;
    }
}
```

✓ You now have:

- ✓ Intent demo with data passing

- ✓ Dynamic image switch on button click

Would you like me to bundle these into a ZIP or help you prepare Viva answers based on these topics?



Thanks! Here is the complete Android Java solution for **Set No. 5** from your uploaded **University of Mumbai Practical Sheet**.

✓ Q1. Design a ListView with TextView for Dynamic Name List

♦ A. `activity\_main.xml`

```
xml

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:padding="20dp"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ListView
        android:id="@+id/nameListView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

</LinearLayout>
```

♦ B. `MainActivity.java`

```
java

package com.example.dynamiclist;

import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    ListView nameListView;
```

```
String[] names = {
    "Aarav", "Priya", "Rohan", "Simran", "Karan",
    "Ishita", "Dev", "Anjali", "Rahul", "Sneha"
};

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    nameListView = findViewById(R.id.nameListView);

    ArrayAdapter<String> adapter = new ArrayAdapter<>(
        this,
        android.R.layout.simple_list_item_1,
        names
    );

    nameListView.setAdapter(adapter);
}
```

◆ C. Output Summary

✅ This app:

- Uses `ListView`
- Dynamically loads a string array (`names`)
- Each item is shown via `TextView` (using default layout)

Would you like to:

- Export this as a **ZIP Android Studio project**?
- Add a **Toast on list item click** for bonus viva marks?